

Travail pratique #3 : Planification d'un réseau de panneaux publicitaires électroniques

1. Objectifs

- Implémenter un type abstrait de données Carte basé sur une représentation par graphe.
- Implémenter un algorithme de recherche d'un arbre de recouvrement.

2. Problématique

Vous devez écrire un programme en Java nommé Tp3.

Ce programme doit rechercher un arbre de recouvrement d'un graphe.

La mairie d'une ville a décidé d'installer des terminaux publicitaires dans un quartier. Elle souhaite relier tous les sites (places publiques) du quartier entre eux via un réseau filaire. Elle vous a fourni les données suivantes :

- La liste des rues les plus visitées du quartier ;
- Le coût associé à l'installation des panneaux sur un segment de rue.

Chaque rue est identifiée par un ensemble de sites. Les rues et les sites sont identifiés par des noms, selon le format suivant : rue0 a b.

rue0 – nom de la rue ; a, b – noms des sites.

Exemple : rue0 a b 4

Les sites du quartier sont reliés entre eux de manière à former un graphe connexe étiqueté, où :

- les nœuds représentent les sites du quartier ;
- les arêtes représentent les segments de rue reliant les sites entre eux ;
- les poids correspondent au coût sur chaque segment.

La ville veut relier, de façon optimale, tous les sites du quartier par un réseau publicitaire, tout en réduisant les dépenses d'installation. L'objectif du TP3 est de fournir un arbre de recouvrement de coût minimum (ARMin) du graphe des sites

d'un quartier, en utilisant l'algorithme de Kruskal. Les arêtes de mêmes poids doivent être traitées selon l'ordre alphanumérique des nœuds de départ ; en cas d'égalité des poids et des nœuds de départ, on utilisera l'ordre alphanumérique des nœuds d'arrivée.

3. Structure du programme

Pour la correction automatique, vous devez respecter la syntaxe d'appel du programme ainsi que ses formats d'entrée et de sortie.

3.1 Syntaxe d'appel

Le programme Tp3 doit pouvoir être lancé en ligne de commande avec la syntaxe suivante.

```
java Tp3 carte.txt arm.txt
```

Le fichier `carte.txt` spécifie une carte des sites d'un quartier ainsi que les données de coût associées à l'installation de panneaux sur chaque rue.

Les résultats produits par votre programme doivent être écrits dans le fichier `arm.txt`.

3.2 Fichier `carte.txt`

Le fichier `carte.txt` est constitué de :

1. Une liste des sites (nœuds). Un site est spécifié par un nom (une chaîne de caractères) ;
2. Trois tirets (---) de séparation ;
3. Une liste des rues pour l'installation de panneaux.
Une rue est spécifiée par : un nom de rue (une chaîne de caractères) ; un deux-points (:) ; une paire de sites représentant un point de départ et un point d'arrivée d'une arête ; un nombre caractérisant le coût d'installation sur ce segment de la rue (arête) ; un point-virgule (;).
4. Trois tirets (---) de fin.

À titre d'exemple, voici le fichier (carte0.txt) :

```
a
b
c
d
e
f
g
h
i
---
rue0 : a b 4 ;
rue1 : a h 8 ;
rue2 : b h 11 ;
rue3 : b c 8 ;
rue4 : c i 2 ;
rue5 : c f 4 ;
rue6 : c d 7 ;
rue7 : d f 14 ;
rue8 : d e 9 ;
rue9 : e f 10 ;
rue10 : f g 2 ;
rue11 : g i 6 ;
rue12 : g h 1 ;
rue13 : h i 7 ;
---
```

3.3 Format de sortie pour Tp3

Le programme Tp3 doit afficher l'arbre couvrant de poids minimum (ARM) de la façon suivante :

Afficher d'abord les sites, un nœud par ligne. Ensuite, afficher les arêtes en ordre croissant alphanumérique des nœuds de départ. Si plusieurs arêtes ont le même nœud de départ, vous devez afficher ces arêtes selon l'ordre croissant alphanumérique des nœuds d'arrivée.

Pour chaque arête, vous devez afficher sur une ligne :

- a) le nom de la rue : rue0
- b) les sommets de l'arête : a b
- c) une étiquette (le poids) : 4

Après avoir affiché toutes les arêtes :

- 1. Trois tirets (---) de séparation ;
- 2. Le coût total de l'arbre sur la dernière ligne.

Exemple de sortie :

```
a
b
c
d
e
f
g
h
i
rue0 a b 4
rue1 a h 8
rue6 c d 7
rue4 c i 2
rue8 d e 9
rue5 f c 4
rue10 g f 2
rue12 h g 1
---
37
```

5. Tests

La correction sera effectuée de manière semi-automatique ; le respect de la syntaxe d'appel et du format de sortie est donc obligatoire. La note sera basée sur les résultats obtenus lors de l'exécution de nos tests.

6. Remise

Vous devez remettre le TP3 au plus tard le **28 juillet à 23h59**. Une pénalité de 33 % par jour de retard sera appliquée, avec un maximum de 3 jours de retard.

1.1 Rapport

Vous devez fournir une analyse de la complexité algorithmique de votre solution :

- Identifiez les principaux facteurs qui influencent le temps d'exécution (indice : la taille du problème).
- Donnez la complexité temporelle en notation grand O, avec une justification.
- Le rapport doit être remis électroniquement sur StudiUM avec le code.

1.2 Remise électronique du code source

.zip, Via StudiUM.

7. ÉVALUATION

Ce travail pratique vaut 10 % de la note finale.

Grille de correction

Critère	Description	Pondération
A.	Respect des directives pour la remise	/ 1
B.	Appréciation générale : structure du programme + qualité du code (découpage du programme, choix des types de données, identificateurs significatifs, lisibilité du code, pertinence des commentaires, etc.).	/ 2

Critère	Description	Pondération
	Encapsulation : respect des principes de l'abstraction ; cachez le maximum de la représentation des objets en rendant le plus d'attributs privés possible.	
C.	Fonctionnement correct : le programme produit les bons résultats. L'efficacité n'est pas directement évaluée. Cependant, elle peut être indirectement évaluée si un programme ne parvient pas à produire des résultats dans des délais raisonnables.	/ 4
D.	Exactitude de l'auto-évaluation : vous déclarez que votre programme fonctionne correctement, partiellement, ou aucunement.	/ 0.5
E.	Analyse de l'algorithme	/ 2.5
	Total :	/ 10

Pour les cas problématiques, jusqu'à 2 points peuvent être retranchés pour la qualité de la présentation.